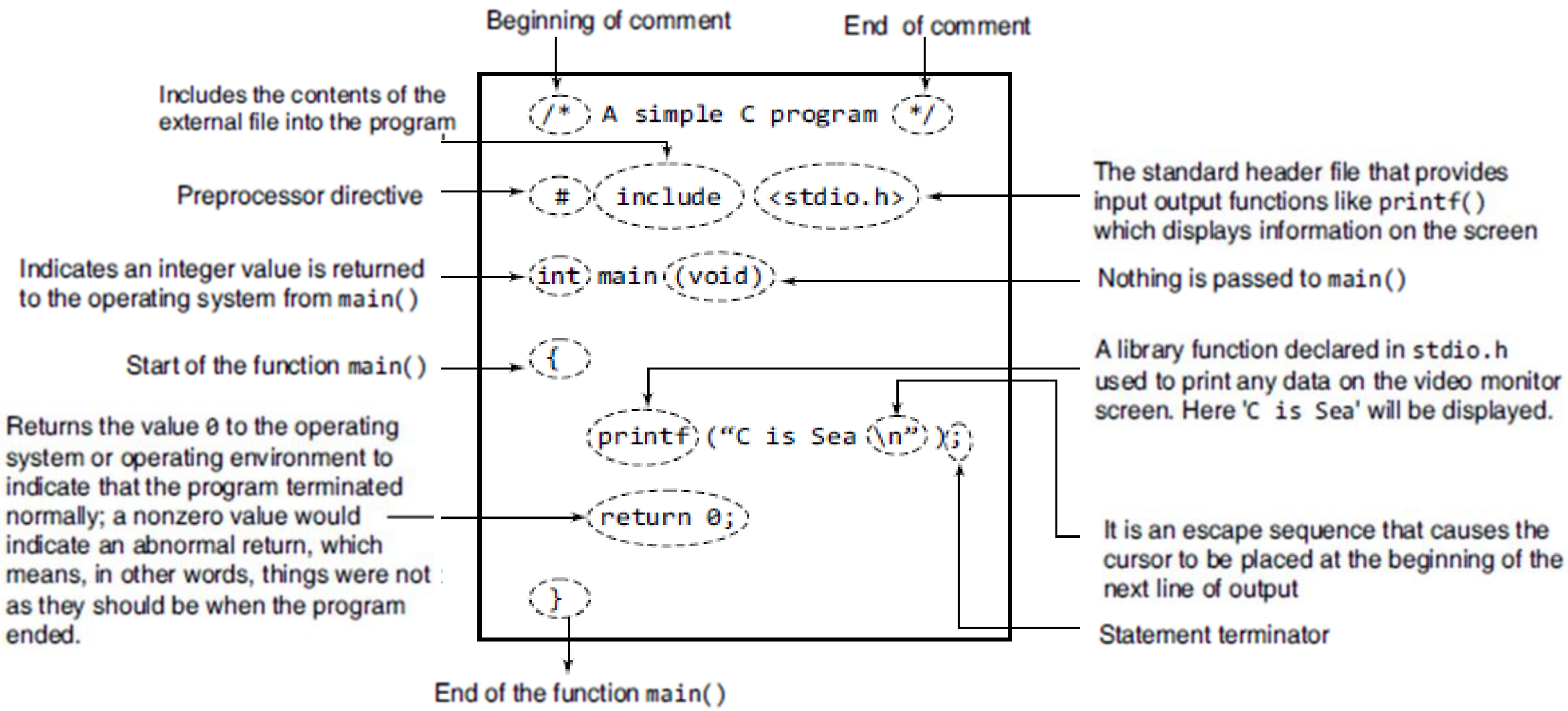
Datatypes

A Review


Beginning of comment
End of comment
Includes the contents of the external file into the program
/* A simple C program */
Preprocessor directive
include <stdio.h>
The standard header file that provides input output functions like printf() which displays information on the screen
Indicates an integer value is returned to the operating system from main()
int main (void)
Nothing is passed to main()
Start of the function main()
{
A library function declared in stdio.h used to print any data on the video monitor screen. Here 'C is Sea' will be displayed.
printf ("C is Sea \n");
Returns the value 0 to the operating system or operating environment to indicate that the program terminated normally; a nonzero value would indicate an abnormal return, which means, in other words, things were not as they should be when the program ended.
return 0;
It is an escape sequence that causes the cursor to be placed at the beginning of the next line of output
}
Statement terminator
End of the function main()

Including files in C program

- Each header file contains one or more function declarations, data type definitions, and macros.

1. #include <filename>
   - Often used to include standard headers
   - Read from the compiler's standard "include" file directory
2. #include "filename"
   - Often used to include nonstandard header files that the programmer creates for use in the program.
   - Tells the preprocessor to look for the file in the current directory first, then look for it in the predefined locations the programmer set up

Difference between preprocessor directives and program statements

1. They must begin in the first column and no spaces are allowed between # and include
2. They are not terminated by a semicolon.

int main(void)

- When a C program is executed, main() is where the action starts.
- **When all instructions of a function have been executed, the function returns control to the calling point, to which it may optionally return the results of its computations.**
- From main() function control returns to the operating system at program termination.

Watcom C/C++, IBM VisualAge C/C++, and Microsoft Visual C/C++ compilers, main() can return **`void/int`**.

MetaWare High C/C++ and EMX C/C++ , the return type of main() has to be declared as **`int`**.

Steps in Learning English

Alphabets → Words → Sentences → Paragraph

Steps in Learning C

Alphabets , digits, Special Symbols → Constants, Variables, Keywords → Instructions → Program

C character constants

A character denotes any alphabet, digit or special symbol used to represent information.

Alphabets	A, B, ….., Y, Z a, b, ……, y, z
Digits	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Special symbols	~ ‘ ! @ # % ^ & * ( ) _ - + = | \ { } [ ] : ; " ' < > , . ? /

The alphabets, numbers and special symbols when properly combined form constants, variables and keywords

- Constants: A constant is an entity that doesn't change

- Variables: A variable can be considered as a name given to the location in memory where this constant is stored. A variable is an entity that may change.

- Keywords: Keywords are the vocabulary of C. Because they are special to C, one cannot use them for variable names.

Structure of a C program

Preprocessor directives

Global declarations

```
int main (void)
{
```

Local definitions

Statements

```
return 0;
}
```

Preprocessor directives tell the preprocessor to look for special code libraries, make substitutions in the code, and in other ways prepare the code for translation into machine language

It is visible to all parts of the program

Describes the data that will be used in the function

Consists of the instructions that cause the computer to do something

Declaration/ Definition

A *declaration* announces the properties of a data object or a function. It facilitates type checking.

A *definition* actually sets aside storage space (in the case of a data object) or indicates the sequence of statements to be carried out (in the case of a function).

Declaration: Describing the type of a data object to the compiler but not allocating any space for it.

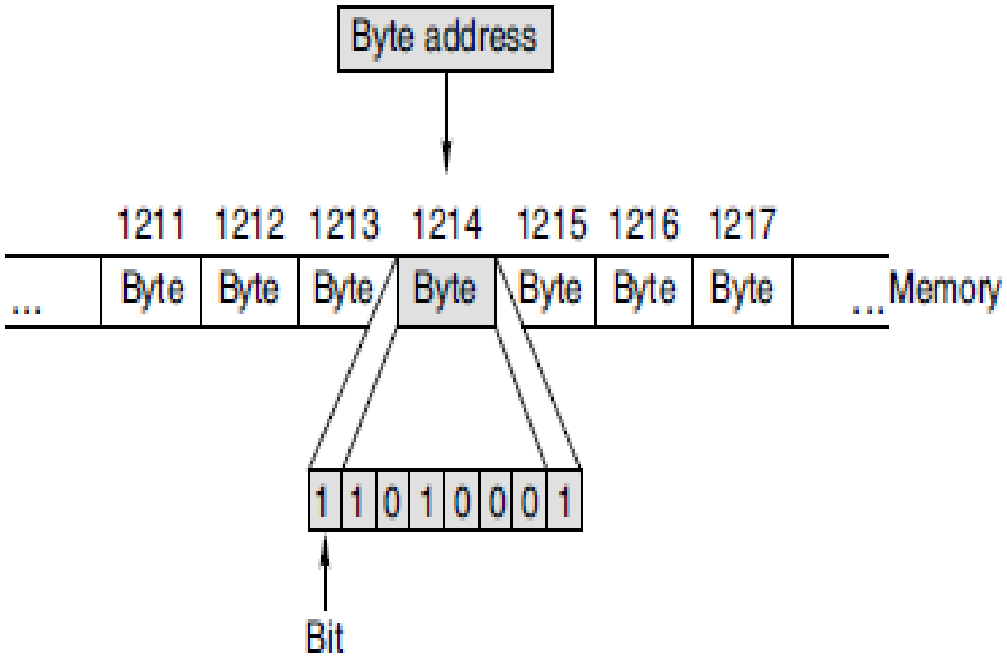
```
int a;
```

Definition :Declaration of a data object and also allocating space to hold the data object.

```
int a=5;
```

Data Object in C

- Data is stored in the form of binary digits 0 and 1 bits (typically 8 bits, 16 bits, 32 bits or, 64 bits) in the memory at physical memory locations. These locations are known as the memory address.



- The amount of bits on which it can operate simultaneously is known as the *word length* of the computer where 1 word= 1byte.

- A 32-bit machine can simultaneously operates on 32 bits of data.

Identifiers

- It is a sequence of characters invented by the programmer to identify or name a specific object, and the name is formed by a sequence of letters, digits, and underscores.

- In C, variables, arrays, functions, and labels are named

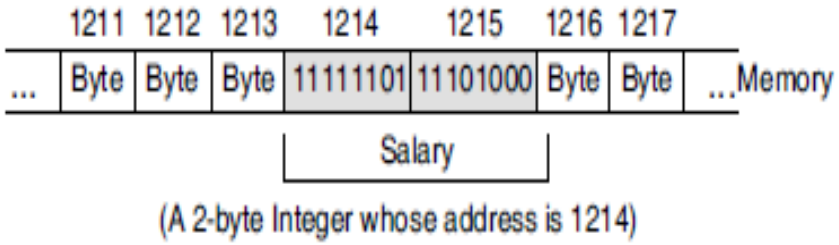
Variables

- A variable is an identifier for a memory location in which data can be stored and can be utilized in various computations in a program.
- Every variable is mapped to a unique memory address.

- Compiler generates an executable code which maps data entities to memory locations during compilation.

- The compiler uses the *address* of the first byte at which the variable is is allocated to refer to it.

```
int salary = 65000;
```



Attributes of a variable

1. Datatype

   Established when the variable is defined, e.g., integer, real, character. Once defined, the type of a C variable cannot be changed.

2. Name :

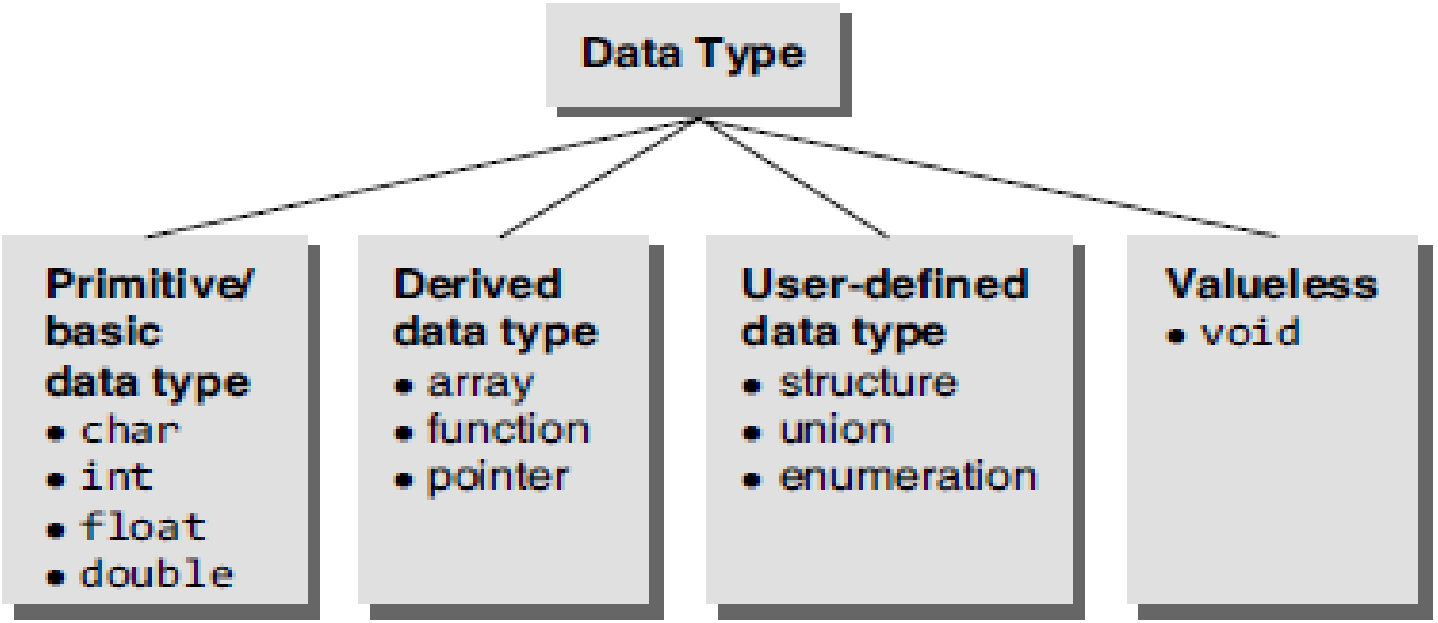
   Can be upto 31 characters in length

   Case sensitive

3. Value:
   - Can be changed by assigning a new value to the variable.
   - The kind of values a variable can assume depends on its type. For example, an integer variable can only take integer values, e.g., 2, 100, –12.

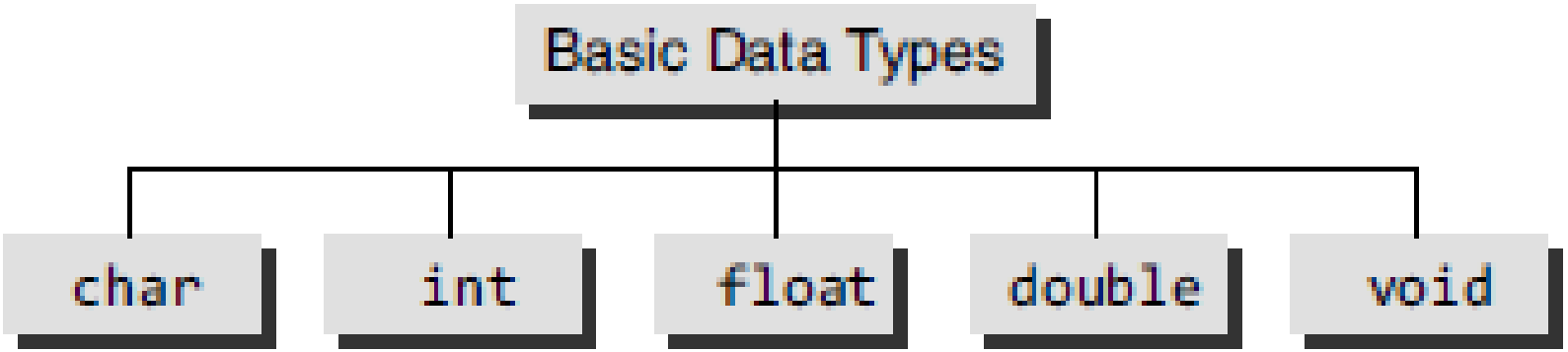
DATA TYPES IN C

- The type or data type of a variable determines the set of values that a variable might take and the set of operations that can be applied to those values.



Basic Datatypes

- Character
- Integer
- Floating-point
- Double precision floating point
- Valueless



Sizes and ranges of basic data types in c for a 16-bit computer

Data type	Size (in bits)	Range
char	8	–128 to 127
int	16	–32768 to 32767
float	32	1.17549×10^{-38} to 3.40282×10^{38}
double	64	2.22507×10^{-308} to 1.79769×10^{308}
void	8	valueless

Sizes and ranges of basic data types in C for a 32-bit computer

Data type	Size (in bits)	Range
char	8	–128 to 127
int	32	–2147483648 to 2147483647
float	32	1.17549×10^{-38} to 3.40282×10^{38}
double	64	2.22507×10^{-308} to 1.79769×10^{308}
void	8	valueless

Size specifiers

- The *size qualifiers* alter the size of the basic data types.
- int: short and long (short <= int <= long)
- Float: long
- Short int tells the C compiler that the particular variable being declared is used to store fairly small integer values.

***Sign specifiers*—signed and unsigned**

- **used only with the basic data types int and char.**

	16-bit machine	32-bit machine	64-bit machine
short int	2	2	2
int	2	4	4
long int	4	4	8

Allowed combinations of basic data types and modifiers in C for a 16-bit computer

Data type	Size (bits)	Range	Default type
`char`	8	–128 to 127	`signed char`
`unsigned char`	8	0 to 255	None
`signed char`	8	–128 to 127	`char`
`int`	16	–32768 to 32767	`signed int`
`unsigned int`	16	0 to 65535	`unsigned`
`signed int`	16	–32768 to 32767	`int`
`short int`	16	–32768 to 32767	`short`, `signed short`, `signed short int`
`unsigned short int`	16	0 to 65535	`unsigned short`
`signed short int`	16	–32768 to 32767	`short`, `signed short`, `short int`
`long int`	32	–2147483648 to 2147483647	`long`, `signed long`, `signed long int`
`unsigned long int`	32	0 to 4294967295	`unsigned long`
`signed long int`	32	–2147483648 to 2147483647	`long int`, `signed long`, `long`
`float`	32	3.4E–38 to 3.4E+38	None
`double`	64	1.7E–308 to 1.7E+308	None
`long double`	80	3.4E–4932 to 1.1E+4932	None

C Keywords

Keywords are the words whose meaning has already been explained to the C compiler (or in a broad sense to the computer). The keywords cannot be used as variable names because if we do so we are trying to assign a new meaning to the keyword, which is not allowed by the computer.

There are only 32 keywords available in C.

e.g. auto, double, int, struct, break, else, long, switch, case, enum, register, typedef, char, etc.

Keywords

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Keywords added in C99 standard

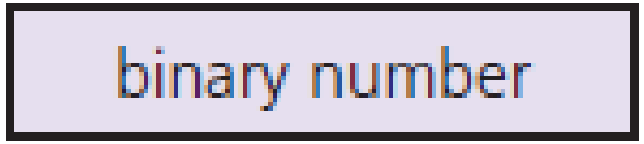
_Bool _Complex _Imaginary inline restrict

Keywords added in C11 draft standard

_Alignas _Alignof _Atomic _Generic _Noreturn _Static_assert _Thread_local

Integer/ Real Number

type `int` format



type `double` format

Integer Number Variables

- When you declare a numeric variable in C, you have two choices: signed and unsigned. Everything is signed unless you specifically type unsigned before the variable type.
- Some C compilers store 0 in newly created numeric variables.
- **int *variable name*;**
- E.g.

```
int anum;
anum=10;

int anum=10;
```

Rules for Constructing Integer Constants

(a) An integer constant must have at least one digit.

(b) It must not have a decimal point.

(c) It can be either positive or negative.

(d) If no sign precedes an integer constant it is assumed to be positive.

(e) No commas or blanks are allowed within an integer constant.

(f) The allowable range for integer constants is -32768 to 32767.

Ex.: 426, +782, -8000, -7605

Decimal Number Variables

- Float and double (also long double)
- For example:

```
float total;
double sum;
total=0.0;
sum=12.50;
```

Rules for Constructing Real Constants

(a) A real constant must have at least one digit.

(b) It must have a decimal point.

(c) It could be either positive or negative.

(d) Default sign is positive.

(e) No commas or blanks are allowed within a real constant.

Ex.: +325.34, 426.0, -32.76, -48.5792

Character Variables

- A single character stored in one byte.
- To assign, or store, a character value in a **char** data type is easy - a character variable is just a symbol enclosed by single quotes.
- A **char** variable is just a single-byte integer variable that can hold a number in the range 0 to 255, which can optionally be interpreted as a character.
- For example
  - **char c;**
  - **c='A'**

Rules for Constructing Character Constants

(a) A character constant is a single alphabet, a single digit or a single special symbol enclosed within single inverted commas. Both the inverted commas should point to the left. For example, ’A’ is a valid character constant whereas ‘A’ is not.

(b) The maximum length of a character constant can be 1 character.

Ex.: 'A'

' I'

'5' '='

Escape sequence

Escape sequence	Description
`\n`	Newline. Position the cursor at the beginning of the next line.
`\t`	Horizontal tab. Move the cursor to the next tab stop.
`\a`	Alert. Produces a sound or visible alert without changing the current cursor position.
`\\`	Backslash. Insert a backslash character in a string.
`\"`	Double quote. Insert a double-quote character in a string.

Format Specifiers(Input/output)

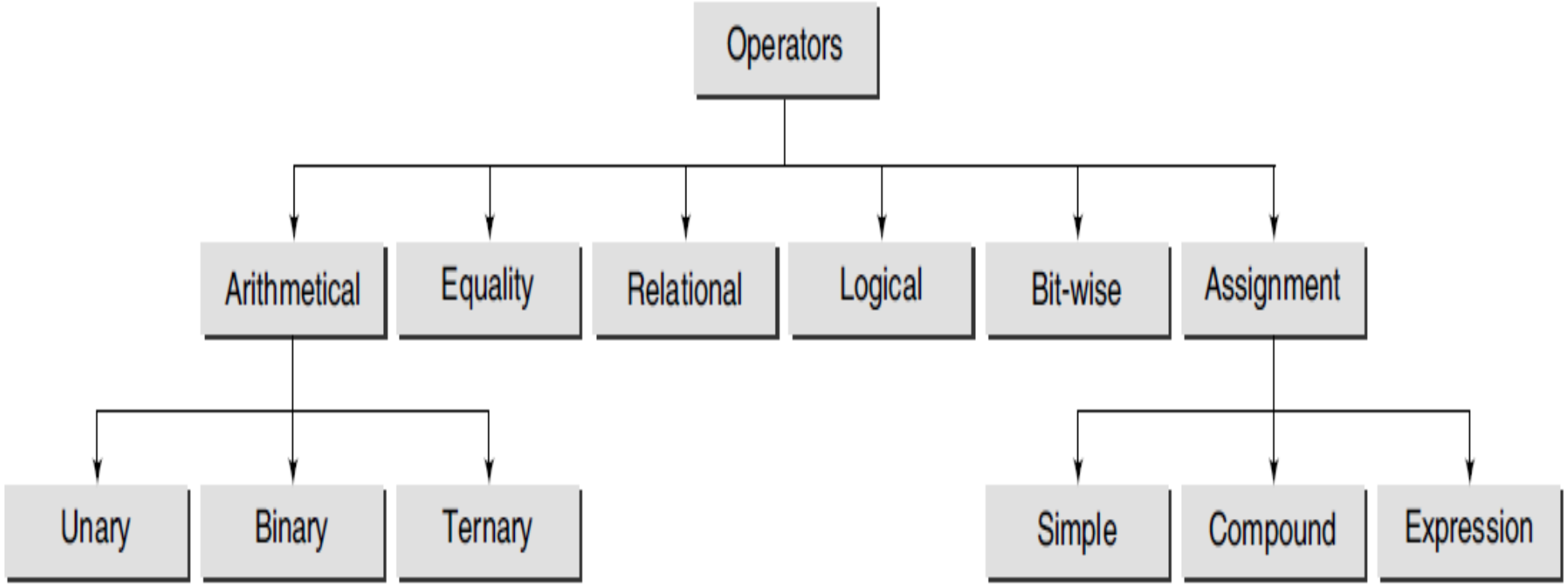
Specifier	Purpose
%c	char single character
%d (%i)	int signed integer
%e (%E)	float or double exponential format
%f	float or double signed decimal
%g(%G)	float or double use %f or %e as required
%o	int unsigned octal value
%p	pointer address stored in pointer
%s	array of char sequence of characters
%u	int unsigned decimal
%x (%X)	int unsigned hex value

printf ("<format string>", <list of variables>) ;

```
printf("The color: %s\n", "blue");
printf("First number: %d\n", 12345);
printf("Second number: %04d\n", 25);
printf("Third number: %i\n", 1234);
printf("Float number: %3.2f\n", 3.14159);
printf("Hexadecimal: %x\n", 255);
printf("Octal: %o\n", 255);
printf("Unsigned value: %u\n", 150);
printf("Just print the percentage sign %%\n", 10);
```

```
The color: blue
First number: 12345
Second number: 0025
Third number: 1234
Float number: 3.14
Hexadecimal: ff
Octal: 377
Unsigned value: 150
Just print the percentage sign %
```

Operator in C



Arithmetic in C

- Algebraic operation:add, subtract, multiply ,divide, remainder.
  - The asterisk (`*`) :multiplication
  - the percent sign (`%`) denotes the remainder operator
- In algebra, to multiply *a* times *b*, we simply place these single-letter variable names side by side as in *ab*.
- In C, however, if we were to do this, `ab` would be interpreted as a single, two-letter name (or identifier).
- Therefore, C requires that multiplication be explicitly denoted by using the `*` operator as in `a * b`.
- The arithmetic operators are all binary operators.
- For example, the expression `3 + 7` contains the binary operator `+` and the operands `3` and `7`.

C operation	Arithmetic operator	Algebraic expression	C expression
Addition	+	$f + 7$	`f + 7`
Subtraction	-	$p - c$	`p - c`
Multiplication	*	bm	`b * m`
Division	/	x / y or $\frac{x}{y}$ or $x \div y$	`x / y`
Remainder	%	$r \bmod s$	`r % s`

Fig. 2.9 | Arithmetic operators.

Arithmetic in C (Cont.)

Integer Division and the Remainder Operator

- Integer division yields an integer result.
- For example, the expression `7 / 4` evaluates to `1` and the expression `17 / 5` evaluates to `3`.
- C provides the remainder operator, `%`, which yields the remainder after **integer division**.
- The remainder operator is an integer operator that can be used only with integer operands.
- The expression `x % y` yields the remainder after `x` is divided by `y`.
- Thus, `7 % 4` yields `3` and `17 % 5` yields `2`.

Arithmetic in C (Cont.)

Arithmetic Expressions in Straight-Line Form

- Arithmetic expressions in C must be written in straight-line form to facilitate entering programs into the computer.
- Thus, expressions such as "`a` divided by `b`" must be written as `a/b` so that all operators and operands appear in a straight line.
- The algebraic notation

$$\frac{a}{b}$$

is generally not acceptable to compilers.

Arithmetic in C (Cont.)

Parentheses for Grouping Subexpressions

- Parentheses are used in C expressions in the same manner as in algebraic expressions.
- For example, to multiply `a` times the quantity `b + c` we write `a * ( b + c )`.

Arithmetic in C (Cont.)

Rules of Operator Precedence

- C applies the operators in arithmetic expressions in a precise sequence determined by the following rules of operator precedence, which are generally the same as those in algebra:
  - Operators in expressions contained within pairs of parentheses are evaluated first. **Parentheses are said to be at the "highest level of precedence**." In cases of nested, or embedded, parentheses, such as
    - `( ( a + b ) + c )`

    the operators in the **innermost pair of parentheses** are applied first.

Arithmetic in C (Cont.)

- **Multiplication, division and remainder** operations are applied next. If an expression contains several multiplication, division and remainder operations, evaluation proceeds from left to right. Multiplication, division and remainder are said to be on the same level of precedence.
- **Addition and subtraction** operations are evaluated next. If an expression contains several addition and subtraction operations, evaluation proceeds from left to right. Addition and subtraction also have the same level of precedence, which is lower than the precedence of the multiplication, division and remainder operations.
- The **assignment operator** (=) is evaluated last.

Associativity in C

The rules of operator precedence specify the order C uses to evaluate expressions. When we say evaluation proceeds from left to right, we're referring to the associativity of the operators.

Operator(s)	Operation(s)	Order of evaluation (precedence)
( )	Parentheses	Evaluated first. If the parentheses are nested, the expression in the *innermost* pair is evaluated first. If there are several pairs of parentheses "on the same level" (i.e., not nested), they're evaluated left to right.
*	Multiplication	Evaluated second. If there are several, they're evaluated left to right.
/	Division	
%	Remainder	
+	Addition	Evaluated third. If there are several, they're evaluated left to right.
-	Subtraction	
=	Assignment	Evaluated last.

Arithmetic in C (Cont.)

- Illustrates the order in which the operators are applied for the given expression:

$$y = 2 * 5 * 5 + 3 * 5 + 7$$

Step 1.	`y = 2 * 5 * 5 + 3 * 5 + 7;` `2 * 5 is 10`	*(Leftmost multiplication)*
Step 2.	`y = 10 * 5 + 3 * 5 + 7;` `10 * 5 is 50`	*(Leftmost multiplication)*
Step 3.	`y = 50 + 3 * 5 + 7;` `3 * 5 is 15`	*(Multiplication before addition)*
Step 4.	`y = 50 + 15 + 7;` `50 + 15 is 65`	*(Leftmost addition)*
Step 5.	`y = 65 + 7;` `65 + 7 is 72`	*(Last addition)*
Step 6.	`y = 72`	*(Last operation—place 72 in y)*

Fig. 2.11 | Order in which a second-degree polynomial is evaluated.

Arithmetic in C (Cont.)

- As in algebra, it's acceptable to place unnecessary parentheses in an expression to make the expression clearer.
- These are called redundant parentheses.

```c
#include <stdio.h>
int main()
  {
    int a;
    int b = 4;
    int c = 8;
    int d = 2;
    int e = 4;
    int f = 2;
    a = b + c / d + e * f;
                  /* result without parentheses */
    printf("The value of a is = %d \n", a);
    a = (b + c) / d + e * f;
                     /* result with parentheses */
    printf("The value of a is = %d \n", a);
    a = b + c / ((d + e) * f);
             /* another result with parentheses */
    printf("The value of a is = %d \n", a);
    return 0;
}
```

```
The value of a is = 16
The value of a is = 14
The value of a is = 4
```

Give the output of the following program:

```
#include <stdio.h>
int main( )
 {
 int x = 3,y = 5,z = 7,w;
 w = x % y + y % x - z % x - x % z;
 printf("%d \n", w);
 w = x / z + y / z + (x + y) / z;
 printf("%d\n", w);
 w = x / z * y / z + x * y / z;
 printf("%d\n", w);
 w = x % y % z + z % y % (y % x);
 printf("%d\n", w);
 w = z / y / y / x + z / y / (y / x);
 printf("%d\n", w);
 return 0;
 }
```

Type conversions in C

- Integer and Float Conversions

1. An arithmetic operation between an integer and integer always yields an integer result.
2. An operation between a real and real always yields a real result.
3. An operation between an integer and real always yields a real result. In this operation the integer is first promoted to a real and then the operation is performed. Hence the result is real.

Examples

Operation	Result	Operation	Result
5 / 2	2	2 / 5	0
5.0 / 2	2.5	2.0 / 5	0.4
5 / 2.0	2.5	2 / 5.0	0.4
5.0 / 2.0	2.5	2.0 / 5.0	0.4

Type Conversion in Assignments

- It may so happen that the type of the expression and the type of the variable on the left-hand side of the assignment operator may not be same. In such a case the value of the expression is promoted or demoted depending on the type of the variable on left-hand side of =.

```
int   i ;
float   b ;
i = 3.5 ;
b = 30 ;
```

```
float   a, b, c ;
int   s ;
s = a * b * c / 100 + 32 / 4 - 3 * 1.1 ;
```

Examples

Arithmetic Instruction	Result	Arithmetic Instruction	Result
k = 2 / 9	0	a = 2 / 9	0.0
k = 2.0 / 9	0	a = 2.0 / 9	0.2222
k = 2 / 9.0	0	a = 2 / 9.0	0.2222
k = 2.0 / 9.0	0	a = 2.0 / 9.0	0.2222
k = 9 / 2	4	a = 9 / 2	4.0
k = 9.0 / 2	4	a = 9.0 / 2	4.5
k = 9 / 2.0	4	a = 9 / 2.0	4.5
k = 9.0 / 2.0	4	a = 9.0 / 2.0	4.5

```c
#include <stdio.h>
int main()
{
  double  x;
  int     i;
  i = 1400;
  x = i;  /* conversion from int to double */
  printf("x = %10.6le i = %d\n",x,i);
  x = 14.999;
  i = x;  /* conversion from double to int */
  printf("x = %10.6le i = %d\n",x,i);
  x = 1.0e+60;    /* a LARGE number */
  i = x;  /* won't fit - what happens ?? */
  printf("x = %10.6le i = %d\n",x,i);
  return 0;
}
```

```
x = 1.445000e+03 i = 1445
x = 1.499700e+01 i = 14
x = 1.000000e+60 i = 2147483647
```

Compute the value of the variables at LHS

(a) g = big / 2 + big * 4 / big - big + abc / 3 ;
(abc = 2.5, big = 2, assume **g** to be a float)

(b) on = ink * act / 2 + 3 / 2 * act + 2 + tig ;
(ink = 4, act = 1, tig = 3.2, assume **on** to be an int)

(c) s = qui * add / 4 - 6 / 2 + 2 / 3 * 6 / god ;
(qui = 4, add = 2, god = 2, assume **s** to be an int)

(d) s = 1 / 3 * a / 4 - 6 / 2 + 2 / 3 * 6 / g ;
(a = 4, g = 3, assume **s** to be an int)

Relational Operators

- C provides six relational operators for comparing numeric quantities.
- Relational operators evaluate to 1, representing the *true* outcome, or 0, representing the *false* outcome.
- The operands of a relational operator must evaluate to a number.
- Characters are valid operands since they are represented by numeric values.

‘A’ < ‘F’

- The relational operators should not be used for comparing strings because this will result in string *addresses* being compared, not string contents.

Operator	Action	Example
==	Equal	`5 == 5 /* gives 1 */`
!=	Not equal	`5 != 5 /* gives 0 */`
<	Less than	`5 < 5.5 /* gives 1 */`
<=	Less than or equal	`5 <= 5 /* gives 1 */`
>	Greater than	`5 > 5.5 /* gives 0 */`
>=	Greater than or equal	`6.3 >= 5 /* gives 1 */`

- <= and >= operators are only supported.
- =< and => are both invalid and do not mean anything.

Logical Operators

- C provides three logical operators for forming logical expressions.
- Like the relational operators, logical operators evaluate to 1 or 0.
- Logical *negation* is a unary operator that negates the logical value of its single operand. If its operand is nonzero, it produces 0, and if it is 0, it produces 1.
- Logical AnD produces 0 if one or both its operands evaluate to 0. Otherwise, it produces 1.
- Logical OR produces 0 if both its operands evaluate to 0. Otherwise, it produces 1.

Operator	Action	Example	Result
!	Logical Negation	`!(5 == 5)`	0
&&	Logical AND	`5 < 6 && 6 < 6`	0
||	Logical OR	`5 < 6 || 6 < 5`	1

- If the left operand yields a false value, the right operand is not evaluated by a compiler in a logical expression using &&.
- If the left operand evaluates true value, the right operand is not evaluated by the compiler in a logical expression with the || operator.
- The operators && and || have left to right associativity. Hence the left operand is evaluated first and, depending on the output, the right operand may or may not be evaluated.

Conditional operator in c (Ternary Operation)

- The conditional operator has three expressions. It has the general form

  expression1 ? *expression2* : *expression3*

- First, *expression1* is evaluated; it is treated as a logical condition.
- If the result is non-zero, then *expression2* is evaluated and its value is the final result. Otherwise, *expression3* is evaluated and its value is the final result.
- Out of the second and the third expressions of the conditional operator, only one is evaluated.

```
int m = 1, n = 2, min;
min = (m < n ? m : n); /* min is assigned a value 1 */
```

Formatting Values of Type int

- Add a number between the % and the d of the %d placeholder in the printf format string.
- This number specifies the **field width**—the number of columns to use for the display of the value.

```
printf("Results: %3d meters = %4d ft. %2d in.\n",
       meters, feet, inches);
```

Indicates that 3 columns will be used to display the value of meters, 4 columns will be used for feet, and 2 columns will be used for inches

If `meters` is `21`, `feet` is `68`, and `inches` is `11`, the program output will be

```
Results:    21 meters =    68 ft. 11 in.
```

Displaying 234 and –234 Using Different Placeholders

Value	Format	Displayed Output	Value	Format	Displayed Output
234	%4d	▮234	-234	%4d	-234
234	%5d	▮▮234	-234	%5d	▮-234
234	%6d	▮▮▮234	-234	%6d	▮▮-234
234	%1d	234	-234	%2d	-234

Formatting Values of Type double

- To describe the format specification for a type double value, we must indicate both the total *field width* needed and the number of *decimal places* desired.
- There will be at least one digit before the decimal point because azero is printed as the whole-number part of fractions that are less than 1.0 and greater than −1.0

format : %*n*.*m*f

where

- *n* is a number representing the total field width
- *m* is the desired number of decimal places

Displaying x Using Format String Placeholder %6.2f

Value of x	Displayed Output	Value of X	Displayed Output
-99.42	-99.42	-25.554	-25.55
.123	▮▮0.12	99.999	100.00
-9.536	▮-9.54	999.4	999.40